

SKILL WEEK-2

Session 3

Task1: To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

(Source code screenshot omitted from this recreation — no username in it.)

Output:

```
sathwika@123:~$ nano session3_task1.c
sathwika@123:~$ ^C
sathwika@123:~$ gcc session3_task1.c -o session3_task1
sathwika@123:~$ ./session3_task1
Session 3 - Command History
Type commands. Use UP/DOWN arrows for history.
Press Ctrl+C to stop.

myshell> hello
You entered: hello
myshell> test
You entered: test
```

Observation: I learned how to use escape sequences to detect special keyboard inputs such as the Up and Down arrow keys. I learned how to store previously entered commands in a history buffer and navigate through them using the arrow keys. I also learned how to update the input buffer when recalling previous commands and manage user input in an interactive shell.

Task2: To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

(Source code screenshot omitted from this recreation — no username in it.)

Output:

```
sathwika@123:~$ nano session3_task2.c
sathwika@123:~$ gcc session3_task2.c -o session3_task2
sathwika@123:~$ ./session3_task2
Session 3 - Dynamic Memory Management
Enter commands. Type 'exit' to finish.
myshell> heloo
Stored command: heloo
Buffer size: 10 bytes
myshell> exit
All dynamically allocated memory released.
```

Observation: I learned how to use dynamic memory allocation in C using malloc() and realloc(). I learned how to resize a buffer when more memory is required and how to store commands using a linked list. I also learned the importance of releasing dynamically allocated memory using free() to prevent memory leaks.

Session 4

Task1: To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

(Source code screenshot omitted from this recreation — no username in it.)

Output:

```
sathwika@123:~$ nano session4_task1.c
sathwika@123:~$ gcc session4_task1.c -o session4_task1
sathwika@123:~$ ./session4_task1
Session 4 - Tokenization
Enter a command: ls -l | grep txt

Tokens identified:
Token 1: [ls]
Token 2: [-l]
Token 3: [grep]
Token 4: [txt]

Token stream validation:
Token stream is valid.
sathwika@123:~$
```

Observation: I learned how to split a command into individual tokens using tokenization. I understood how whitespace and special delimiters such as | can separate different parts of a command. I also learned how to store tokens in a token structure and validate the resulting token stream before further processing.

Task2: To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

(Source code screenshot omitted from this recreation — no username in it.)

Output:

```
sathwika@123:~$ nano session4_task2.c
sathwika@123:~$ gcc session4_task2.c -o session4_task2
sathwika@123:~$ ./session4_task2
Session 4 - Parser
Enter a command: ls -l | grep txt

Parse Structure:
Command: ls

Command: ls
Arguments:
  Argument 1: -l
  Argument 3: grep
  Argument 4: txt

Parse tree / execution structure:
ROOT
├── COMMAND: ls
│   ├── ARGUMENT: -l
│   ├── PIPE
│   ├── ARGUMENT: grep
│   └── ARGUMENT: txt
└──

Syntax validation: SUCCESS
Execution structure generated successfully.
sathwika@123:~$
```

Observation: I learned how to design basic parser logic for processing user commands. I understood how to validate command syntax, detect errors such as invalid or consecutive pipe symbols, and handle empty commands. I also learned how tokens can be organized into a parse structure containing commands, arguments, and pipes, which can later be used for command execution.

